TP2 — Orquestração K3s, Proxy Redundante (HA) e Persistência de Estado Não Destrutiva

DCC091A — Tópicos em Redes e Processamento Distribuído I

Grupo 6

10 de novembro de 2026

1 Objetivo

Evoluir a infraestrutura do TP1 para um cluster K3s de dois nós protegido por dois proxies Nginx em alta disponibilidade (Keepalived com VIP flutuante 192.168.56.254), com uma aplicação web e um banco de dados persistente, e demonstrar que o ambiente pode ser pausado (`vagrant suspend`/`halt`) e retomado sem reprovisionamento.

2 Arquitetura

VM	Nome	IP	vCPU/RAM	Papel
1	bastion	.10	1 / 1024 MB	Nó de controle Ansible, kubectl, build da imagem, Nginx + Keepalived MASTER (101)
2	k3s-server	.20	2 / 2048 MB	K3s server: control plane, datastore, CoreDNS, local-path
3	k3s-worker	.30	2 / 2048 MB	K3s agent
4	auxiliary	.40	1 / 1024 MB	Nginx + Keepalived BACKUP (100), backup em `./data_backup`

Tabela 1: Máquinas virtuais (Ubuntu 22.04, rede host-only 192.168.56.0/24, interface `enp0s8`).

Fluxo de uma requisição: cliente → VIP 192.168.56.254:80 (proxy que detém o VIP) → upstream `least_conn` para 192.168.56.20:30080 e 192.168.56.30:30080 (NodePort) → kube-proxy → uma das duas réplicas da API → PostgreSQL (Service ClusterIP `postgres`).

2.1 Aplicação

API em Go (`app/main.go`) com as rotas `/info` (Pod e nó que atenderam, total de registros), `/health` (200 se o banco responde, usada nas probes) e `/records` (GET lista, POST grava). O Dockerfile é multi-stage: compila em `golang:1.22-alpine` e copia apenas o binário estático para uma imagem `FROM scratch`, executada como usuário não-root. A imagem final tem **5,73 MB** (limite do roteiro: 100 MB).

A imagem é gerada com `podman` no bastion, exportada como `docker-archive` e importada no containerd de cada nó (`k3s ctr images import`), com `imagePullPolicy: Never`. Assim não é preciso um registry.

2.2 Manifestos (k3s/)

- `01-configmap-secret.yaml`: ConfigMap `app-config` (ambiente, porta, host, nome e usuário do banco) e Secret `db-credentials` (senha).
- `02-pvc.yaml`: PVC `postgres-data`, 1 Gi, `local-path`, RWO.
- `03-deployment.yaml`: PostgreSQL (1 réplica, estratégia `Recreate`, PVC) e API (2 réplicas, `topologySpreadConstraints` por nó, liveness e readiness em `/health`, nome do Pod e do nó injetados pela Downward API).
- `04-service.yaml`: `postgres` (ClusterIP) e `tp2-api` (NodePort 30080).

2.3 Automação (Ansible a partir do bastion)

O `Vagrantfile` define o bastion por último: ele recebe as chaves privadas das outras VMs (`.vagrant/machines`) e executa o `site.yml` via `ansible_local`.

- **common**: pacotes (curl, htop, tcpdump, dnsutils, jq), fuso America/Sao_Paulo, `/etc/hosts` com todos os nós e o VIP, `ip_forward` e UFW (entrada negada por padrão; regras por papel).
- **proxy_ha**: Nginx com upstream para os NodePorts, Keepalived com VRRP unicast, `check_nginx.sh` e `ip_nonlocal_bind`.
- **k3s_server**: `/etc/rancher/k3s/config.yaml` (IP e flannel em `enp0s8`, Traefik e ServiceLB desativados), instalação na versão fixa v1.31.4+k3s1, extração do token e exportação do kubeconfig para o bastion.
- **k3s_agent**: instalação do agent e junção ao server com o token extraído.
- **app_deploy**: build da imagem (só quando o código muda), distribuição, `kubectl apply` e espera pelo rollout.
- **backup_sync** (extra): timer na VM 4 que grava `/records` em `/var/lib/app_data` (synced_folder `./data_backup`) a cada minuto.

3 Análise da proposta inicial

A primeira versão do grupo acertava a topologia e os parâmetros, mas tinha problemas que impediriam o funcionamento ou os testes, e que foram corrigidos:

- a interface `eth1` não existe na box `ubuntu/jammy64` (o nome é `enp0s8`), então o VIP e o flannel não subiriam;
- o Ansible seria executado no host com a chave insegura global do Vagrant, mas o Vagrant gera uma chave por VM e o roteiro pede o Ansible no bastion;
- a aplicação (`http-echo`) não tinha build multi-stage, banco de dados nem as rotas `/health` e `/info`;
- o PVC `local-path` RWO montado nas duas réplicas as prende ao mesmo nó;
- o UFW era instalado sem regras, o kubeconfig não era exportado, e o `creates` do agent apontava para um arquivo inexistente (sem idempotência).

4 Problemas encontrados durante a validação

- **Ativação do UFW travava o playbook.** Na primeira ativação o conntrack passa a valer e a conexão SSH já aberta pelo Ansible nos nós K3s era descartada. A tarefa passou a rodar em `async`, seguida de `meta: reset_connection` e `async_status`.

- **Failover em ∼4 s.** Com `weight -20` o master caía para prioridade 81, mas o BACKUP descarta anúncios de prioridade menor e só assume após o `master_down_interval`. Sem `weight`, a falha do script coloca o master em FAULT, ele solta o VIP e anuncia prioridade 0: o tempo caiu para ∼1 s.
- **Idempotência do Secret.** Com `stringData`, o `kubectl apply` sempre responde `configured`; o Secret passou a usar `data` (base64).
- **Portas 80/443 abertas nos nós K3s** pelo Traefik e pelo ServiceLB embutidos, que não são usados; foram desativados.

5 Resultados dos testes

Todos os resultados abaixo foram obtidos em um ambiente recriado do zero (`vagrant destroy -f && vagrant up`, 11 min 46 s, sem falhas) em um host Windows 11 com VirtualBox 7.2.16 e Vagrant 2.4.9.

Teste	Resultado	Status
Idempotência	2ª execução do `site.yml`: `changed=0, failed=0` nas 4 VMs	OK
A — VIP e balanceamento	10 consultas a `/info`: 6 na réplica do k3s-worker e 4 na do k3s-server	OK
B — failover do proxy	Nginx parado às 13:44:49,40; bastion em FAULT às 49,88; VM 4 MASTER às 50,50 (**1,09 s**). 1 de 48 requisições perdida (a que estava em curso). VIP volta ao bastion quando o Nginx retorna	OK
C — suspend/resume	API de volta pelo VIP em **51 s**; registro gravado antes preservado	OK
C — halt/up	API de volta em 108 s; pods, VIP e registros restabelecidos sem reprovisionar	OK
D — reconciliação	Pod apagado recriado, 2/2 réplicas prontas em **4 s**	OK
Backup (VM 4)	`records.json` e `backup.log` atualizados no host a cada minuto	OK

Tabela 2: Resumo da validação.

Trechos de saída

Teste A:

```
13:44:32 proxy=bastion {"node":"k3s-worker","pod":"tp2-api-7598cc9988-rpg9f",...}
13:44:33 proxy=bastion {"node":"k3s-server","pod":"tp2-api-7598cc9988-ht7g2",...}
13:44:34 proxy=bastion {"node":"k3s-worker","pod":"tp2-api-7598cc9988-rpg9f",...}
```

Teste B (logs do Keepalived):

```
13:44:49.401 bastion systemd: Stopped A high performance web server ...
13:44:49.884 bastion Keepalived_vrrp: (VI_1) Entering FAULT STATE
13:44:50.496 auxiliary Keepalived_vrrp: (VI_1) Entering MASTER STATE
13:45:15.499 auxiliary Keepalived_vrrp: (VI_1) Entering BACKUP STATE
```

Teste C (suspend/resume):

```
>>> gravando: registro-teste-c-1794329137
>>> API respondendo pelo VIP 51s apos o inicio da retomada
>>> comando vagrant concluido em 80s
OK: registro 'registro-teste-c-1794329137' preservado
```

6 Limitações

- **Tempo do `vagrant resume`.** O cluster retoma de imediato, mas o Vagrant retoma as VMs uma por vez e espera o SSH de cada uma; o comando inteiro leva 70–80 s. O serviço volta pelo VIP antes disso (51–57 s nas medições), com pouca folga em relação aos 60 s do roteiro e dependente do desempenho do host.
- **NodePort acessível da sub-rede.** O kube-proxy insere suas regras de DNAT antes das do UFW, então a porta 30080 dos nós K3s responde a qualquer host de 192.168.56.0/24, não só aos proxies. O ponto de entrada documentado é o VIP.
- **Banco em um único nó.** Com `local-path`, o volume do PostgreSQL pertence ao nó onde foi criado; se esse nó cair, o banco fica indisponível até ele voltar. Os dados ficam no disco da VM e também são copiados em JSON para o host pela VM 4.

7 Como reproduzir

```
cd tp2
vagrant up
bash scripts/teste-a-balanceamento.sh 10
bash scripts/teste-b-failover.sh
bash scripts/teste-c-ciclo-de-vida.sh # suspend/resume
bash scripts/teste-c-ciclo-de-vida.sh halt # halt/up
bash scripts/teste-d-reconciliacao.sh
vagrant provision bastion # idempotencia: changed=0
```